

MONITORIA TI APRESENTA ≡
LÓGICA DE PROGRAMAÇÃO EM JAVA

AULA 02:

LÓGICA DE PROGRAMAÇÃO EM JAVA



ANTES DE COMEÇAR

QUEM SOMOS?





Jess



Victor



Gabriel



Kauan



Polverari



Matheus



CONSTANTES



- **Variáveis cujo valor não pode ser alterado depois de declarado**
- **Para serem declaradas precisam de um nome, um tipo e um valor associado à variável**
Ex: `int numero = 67`

OPERADORES ARITMÉTICOS



- São problemas que podem ser resolvidos por um computador utilizando uma sequência de padrões e comandos
- Essa sequência de comandos pode ser chamada de **Algoritmo**

OPERAÇÕES:



Aritméticas

Realizam cálculos matemáticos entre valores.

```
a + b    // soma
a - b    // subtração
a * b    // multiplicação
a / b    // divisão
a % b    // resto da divisão
```



Relacionais

Comparam valores e retornam um resultado booleano.

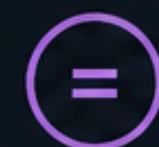
```
a == b    // igual
a != b    // diferente
a > b     // maior que
a < b     // menor que
a >= b    // maior ou igual
a <= b    // menor ou igual
```



Lógicas

Combinam condições e retornam um valor booleano.

```
a && b    // e (and)
a || b    // ou (or)
!a        // negação (not)
```



Atribuição

Armazenam um valor em uma variável.

```
a = b      // atribuição
a += b     // soma e atribui
a -= b     // subtrai e atribui
a *= b     // multiplica e atribui
a /= b     // divide e atribui
a %= b     // resto e atribui
```



Incremento e Decremento

Aumentam ou diminuem o valor de uma variável em 1.

```
a++       // pós-incremento
++a       // pré-incremento
a--       // pós-decremento
--a       // pré-decremento
```

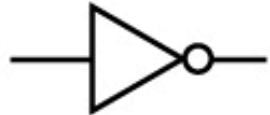


Outras operações

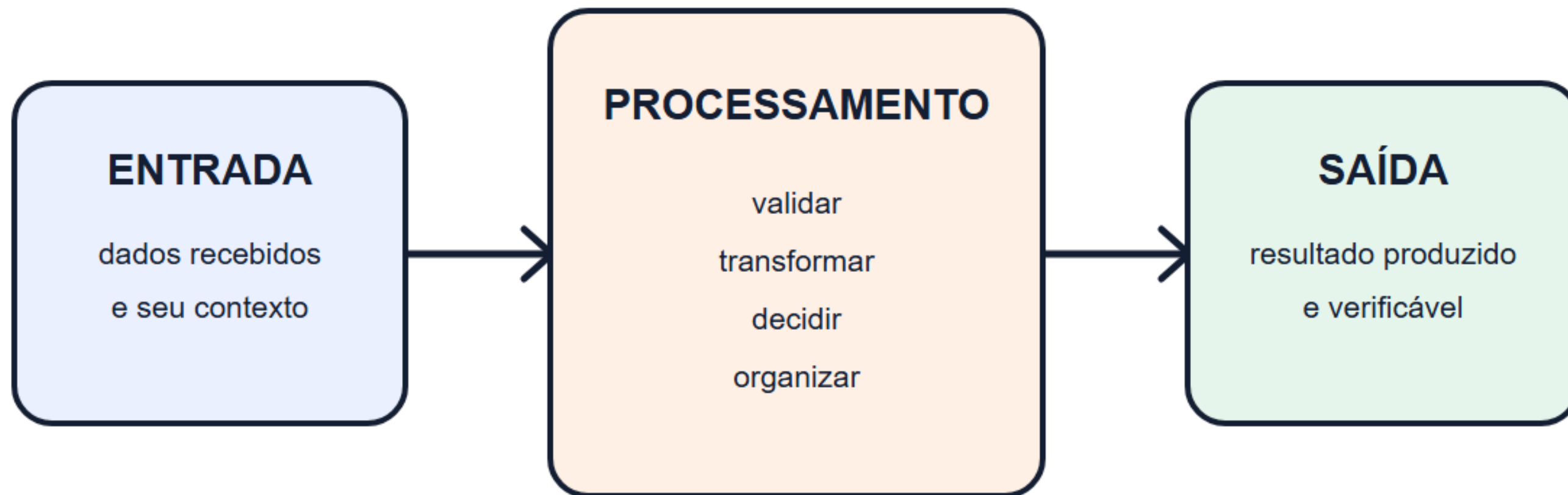
Existem ainda operações específicas, como o operador ternário e o bit a bit.

```
cond ? valor1 : valor2  // ternário
~a                      // negação bit a bit
a & b                   // e bit a bit
a | b                   // ou bit a bit
a ^ b                   // xor bit a bit
a << b                  // deslocamento à esquerda
a >> b                  // deslocamento à direita
```

OPERAÇÕES BOOLEANAS:

Operador Booleano	Nome Padrão	Símbolo Padrão	Função Booleana
E	AND		$A \cdot B$
OU	OR		$A + B$
NÃO	NOT		$\overline{A}$
OU EXCLUSIVO	XOR		$A \oplus B$
NÃO E	NAND		$\overline{A \cdot B}$
NÃO OU	NOR		$\overline{A + B}$
NÃO OU EXCLUSIVO	XNOR		$\overline{A \oplus B}$

ENTRADA, PROCESSAMENTO E SAÍDA



SCANNER:

```
import java.util.Scanner;

public class ExemploEntrada {
    public static void main(String[] args) {
        // Instancia o Scanner apontando para a entrada do teclado (System.in)
        Scanner scanner = new Scanner(System.in);

        // 1. Recebendo texto (String)
        System.out.print("Digite o seu nome: ");
        String nome = scanner.nextLine();

        // 2. Recebendo número inteiro (int)
        System.out.print("Digite a sua idade: ");
        int idade = scanner.nextInt();

        // 3. Recebendo número decimal (double)
        System.out.print("Digite sua altura (ex: 1,75): ");
        double altura = scanner.nextDouble();

        // Exibindo os valores armazenados nas variáveis
        System.out.println("\n--- Dados Armazenados ---");
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade + " anos");
        System.out.println("Altura: " + altura + "m");

        // Fecha o scanner ao finalizar a leitura
        scanner.close();
    }
}
```

LEITURAS E CONVERSÕES

Imagine que você está conversando com o computador e enviando mensagens para ele.

Quando você digita algo e aperta Enter, o Java precisa ler essa informação. O jeito mais seguro de fazer isso e evitar erros é usar o `nextLine()`.

O que cada comando faz?

- `nextLine()`: Ele pega tudo o que você digitou na linha de uma vez só (letras, números e o próprio Enter) e transforma em um texto comum (uma `String`).
- `Integer.parseInt()`: Ele pega esse texto que você leu e o transforma em um número inteiro (sem vírgula).
- `Double.parseDouble()`: Ele pega esse texto e o transforma em um número quebrado (com ponto decimal).

Por que?

Se você usar comandos específicos de números (como `nextInt()`), o Java pega o número, mas deixa o "Enter" solto na memória.

Na próxima vez que você tentar ler um texto, o Java vai achar que aquele "Enter" antigo já era a resposta e vai pular a leitura, deixando você sem conseguir digitar.

Lendo tudo como texto primeiro com `nextLine()`, você limpa a mesa e não deixa nenhum "Enter" sobrando. Depois, você só traduz o texto para número.

EXEMPLO:

```
// Lendo o primeiro número como texto e convertendo
System.out.print(s: "Digite o primeiro número: ");
String texto1 = scanner.nextLine();
int numero1 = Integer.parseInt(texto1);

// Lendo o segundo número como texto e convertendo
System.out.print(s: "Digite o segundo número: ");
String texto2 = scanner.nextLine();
int numero2 = Integer.parseInt(texto2);

// Fazendo a conta com os números de verdade
int resultado = numero1 + numero2;

// Mostrando o resultado na tela
System.out.println("A soma dos dois números é: " + resultado);
```

TERMINAL:

```
Digite o primeiro número: 5
Digite o segundo número: 16
A soma dos dois números é: 21
```

TIPOS DE ERRO:



- **Compilação**
- **Execução(Runtime)**
- **Lógica**

ERRO DE COMPILAÇÃO:

Esse erro acontece antes do programa iniciar, enquanto você está escrevendo o código.

O Java funciona como um vigia gramatical estrito: se você esquecer uma regra da linguagem, **ele nem deixa o programa ser executado**.

- **O que causa:** Erros de digitação, esquecer o ponto e vírgula (;), fechar parênteses errado ou tentar mudar uma constante final.
- **Exemplo:** É como tentar enviar uma carta, mas esquecer de colocar o endereço no envelope. O correio nem aceita a carta.

ERRO DE LÓGICA

O programa roda perfeitamente do início ao fim, não aparece nenhuma mensagem de erro na tela e ele não fecha sozinho.

Porém, o resultado final está errado. O computador fez exatamente o que você mandou, mas você mandou a coisa errada.

- **O que causa:** Errar uma fórmula matemática ou confundir os sinais.
- **Exemplo:** É como pedir um GPS para ir à praia, mas digitar o endereço do trabalho. Você vai chegar a um destino sem problemas, mas não era onde você queria ir.

ERRO DE EXECUÇÃO:

O seu código foi escrito certinho e o programa começou a rodar. Porém, durante o funcionamento, acontece algo inesperado que o Java não sabe como resolver, fazendo o programa fechar sozinho, o famoso crash.

- **O que causa:** Tentar dividir um número por zero.
- **Exemplo:** É como seguir uma receita perfeitamente, mas no meio do caminho o gás da cozinha acabar. O plano era bom, mas algo deu errado no meio da ação.

ERRO DE COMPILAÇÃO:

```
//ERRO DE COMPILAÇÃO
System.out.println(x: "Digite um número e vou analisar se ele é par ou ímpar")
int num1=sc.nextInt();
if (num1%2==0){
    System.out.println("O número "+num1+" é par.")
}
else{
    System.out.println("O número "+num1+" é ímpar")
}
//Faltando ;
```

ERRO DE EXECUÇÃO:

```
//ERRO DE EXECUÇÃO
System.out.println(x: "Digite três números e vou exibir a média entre eles.");

System.out.print(s: "Primeiro número: ");
double nummed1=sc.nextDouble();

System.out.print(s: "Segundo número: ");
double nummed2=sc.nextDouble();

System.out.print(s: "Terceiro número: ");
double nummed3=sc.nextDouble();

System.out.println("A media entre eles é: "+(nummed1+nummed2+nummed3)/0);
//DIVISÃO POR 0
```

ERRO DE LÓGICA

```
//ERRO DE LÓGICA
double nummul5=sc.nextDouble();
System.out.print(s: "Digite um número e vou verificar se ele é múltiplo de 5: ");
else{
    System.out.println("O número "+nummul5+" não é múltiplo de 5.");
}
if (nummul5%5==0){
    System.out.println("O número "+nummul5+" é múltiplo de 5.");
}
//Colocar o else antes do if
//Scanner antes do print
```

FLUXO DE EXECUÇÃO:

É a ordem em que o computador lê e executa as linhas de código do seu programa.

Imagine o fluxo como um rio, a água corre em uma direção, mas nós podemos colocar barreiras para mudar o seu caminho. No Java existem 3 formas de controlar esse fluxo:

1. Fluxo Sequencial (o padrão)

É o fluxo passo a passo. O Java executa a linha 1, depois a linha 2, depois a linha 3, sem pular nada.

2. Fluxo Condicional (Desvios no caminho)

Às vezes, o programa precisa tomar uma decisão e seguir caminhos diferentes. Usamos o if (se) e o else (senão) para isso.

O fluxo se divide: ou o computador vai para um lado, ou vai para o outro.

3. Fluxo de Repetição (Laços ou Loops)

Às vezes, queremos que o fluxo volte para trás e repita o mesmo pedaço de código várias vezes. Usamos comandos como while (enquanto) ou for (para).

O fluxo fica "andando em círculos" até uma condição mandar ele parar.

PERGUNTAS? COMENTÁRIOS?

Fique à vontade para compartilhar sua opinião.

